

Class Imbalance in R: Oversampling, SMOTE, and Undersampling Explained

Dataset Balancing: The Class Imbalance Problem

Class imbalance arises when positive examples are rare among a large number of negative examples — a recurring situation in medical diagnosis, fraud detection, leak monitoring, and fault identification in systems.

Traditional machine learning methods assume balanced distributions and symmetric error costs, making them inadequate for these scenarios. To overcome this limitation, two preprocessing strategies stand out: **oversampling**, which increases the representation of the minority class, and **undersampling**, which reduces the influence of the majority class. The choice between them depends on the context and characteristics of the data, as illustrated by the practical examples below.

Oversampling

Resampling data from the positive (minority) class, creating copies or generating new synthetic examples to increase its representation.

- Increases the size of the dataset
- Can lead to overfitting if poorly applied
- Techniques like SMOTE generate synthetic examples

Undersampling

Random elimination of tuples from the negative (majority) class to balance the proportion between classes.

- Reduces the size of the dataset
- May lose important information
- Faster for training models
- Risk of losing relevant patterns

📖 **References:** Zhou and Liu (2006) developed methods for training cost-sensitive neural networks. Tharwat and Schenck (2020) proposed active learning approaches for imbalanced data.

Oversampling and Undersampling in Practice

Scenario: Imbalanced Iris Dataset

Having understood the concepts of oversampling and undersampling, let's see how these techniques apply in practice — using an imbalanced version of the classic Iris dataset as an illustrative example.

Consider that the Iris dataset had the following imbalanced distribution:

Setosa

20 observations

Versicolor

50 observations

Virginica

11 observations

How Do the Techniques Address the Imbalance?

Oversampling Approach

Would increase the minority classes (setosa and virginica) through:

- Replication of existing examples
- Generation of synthetic examples
- Interpolation between existing points
- Result: ~50 observations per class

Undersampling Approach

Would reduce the majority class (versicolor) through:

- Random selection of a subset
- Removal of redundant examples
- Preservation of informative examples
- Result: ~15-20 observations per class

The choice between oversampling and undersampling depends on the initial dataset size, available computational resources, and the importance of preserving all patterns present in the original data — and in the following slides we will see, step by step, how to implement each of these approaches in R.

Overview – Balancing

After understanding how oversampling and undersampling act on class distribution, it's time to see how to implement these strategies in practice. The `daltoolbox` library offers two dedicated functions for this purpose, each reflecting one of the approaches discussed earlier.

`bal_oversampling`

Oversampling of the minority class by random replication.
Increases the dataset without losing existing instances; risk of overfitting if misapplied.

`bal_subsampling`

Undersampling of the majority class by random removal.
Reduces the dataset to the size of the minority class; faster, but may lose relevant information.

- ❏ **In the next slides:** We will see, step by step, how to configure and apply each of these functions in R — from defining the class column to verifying the final result. Remember: balancing should be applied **only on the training set**; applying it to the test set distorts the evaluation metrics.

Oversampling – Theory

Oversampling is a fundamental technique in handling class imbalance, where the representation of the minority class is increased to prevent predictive models from being biased towards the majority class. This method aims to balance the dataset by either replicating existing instances or generating new synthetic examples, ensuring the model has sufficient data points for all classes during training.

Two Main Strategies

Random Oversampling

This straightforward approach randomly replicates existing instances of the minority class until a desired class distribution is achieved. While simple and quick to implement, it carries the risk of overfitting, as it creates exact duplicates that do not introduce any new information to the model.

- **Pros:** Easy to understand and implement.
- **Cons:** Increased overfitting risk due to exact copies.

SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE generates synthetic examples rather than just replicating existing ones. It does this by taking a minority class sample and introducing synthetic examples along the line segments joining any of the k minority class nearest neighbors. This approach creates more diverse data, helping to mitigate the overfitting issues seen with random oversampling.

- **Pros:** Generates diverse data, reduces overfitting.
- **Cons:** Computationally more intensive, can introduce noise if neighbors are poorly chosen.

A key trade-off with oversampling is the increased size of the dataset. While beneficial for class balance, this can lead to longer training times for models and, if not applied judiciously (especially with random oversampling), can still induce overfitting by making the model too specific to the duplicated instances.

📄 **Reference:** Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. — the foundational paper introducing SMOTE, essential for deeper understanding of synthetic oversampling.

Oversampling – bal_oversampling

Starting with the first function presented in the overview, see how to apply oversampling in practice with two distinct strategies.

```
library(daltoolbox)
# Artificially imbalanced dataset
iris_imb <- datasets::iris[
  c(1:50, 51:71, 101:111), ]
table(iris_imb$Species)
# setosa versicolor virginica
#   50    21    11

# Random oversampling
bal_r <- bal_oversampling(
  "Species", method = "random", seed = 123)
iris_r <- transform(bal_r, iris_imb)
```

```
table(iris_r$Species)
# setosa versicolor virginica
#   50    50    50

# Oversampling via SMOTE (k=3 neighbors)
bal_s <- bal_oversampling(
  "Species", method = "smote",
  k = 3, seed = 123)
iris_s <- transform(bal_s, iris_imb)
table(iris_s$Species)
# setosa versicolor virginica
#   50    50    50
```

Step by step

- Create an imbalanced dataset: 50 setosa, 21 versicolor, 11 virginica
- `method = "random"`: replicates minority instances until they match the majority
- `method = "smote"`: generates synthetic instances by interpolating between neighbors (`k=3`)
- Both result in 50 instances per class

 **Key concept:** SMOTE creates synthetic examples by interpolating between real minority instances, reducing the risk of overfitting compared to pure replication. In the next slide, we will look at the opposite strategy: instead of increasing the minority, `bal_subsampling` reduces the majority — a faster approach, but with different trade-offs.

Undersampling – Theory

Undersampling is a technique designed to address class imbalance by reducing the representation of the majority class. This is achieved by selectively removing instances from the prevalent class until the dataset's class distribution becomes more balanced, often matching the size of the minority class.

The primary strategy employed in `daltoolbox` for undersampling is **Random Undersampling**, where instances from the majority class are randomly selected and removed without replacement. This method is straightforward and effective in reducing computational load.

Key characteristics of undersampling:

- Reduces the total size of the dataset, leading to faster model training.
- Risk of losing relevant information or patterns from the majority class if important instances are removed.
- Best suited for large datasets where the loss of some majority class data is tolerable and does not significantly impair model performance.

While oversampling adds data to balance classes, undersampling removes it. Both aim for equilibrium but with different trade-offs in terms of data retention, computational cost, and potential for information loss or overfitting.

Pros: Efficiency & Simplicity

Significantly reduces dataset size, leading to faster training times and lower computational resource demands. Easy to implement and understand.

Cons: Information Loss

Risks discarding potentially valuable information or unique patterns present in the removed majority class instances, which can negatively impact model generalization.

📄 **Reference:** Liu, X. Y., Wu, J., & Zhou, Z. H. (2009). Exploratory Undersampling for Class-Imbalance Learning. *IEEE Transactions on Systems, Man, and Cybernetics*, 39(2), 539–550. — a key reference on undersampling strategies and their impact on learning performance.

Undersampling

If oversampling increases the minority, the complementary approach goes in the opposite direction: `bal_subsampling` reduces the majority classes, balancing the dataset in a leaner way.

```
library(daltoolbox)
# Imbalanced dataset
iris_imb <- datasets::iris[
  c(1:50, 51:71, 101:111), ]
table(iris_imb$Species)
# setosa versicolor virginica
#  50    21    11

# Random undersampling
bal <- bal_subsampling("Species", seed = 123)
iris_bal <- transform(bal, iris_imb)
```

```
table(iris_bal$Species)
# setosa versicolor virginica
#   11     11     11

head(iris_bal)
# Sepal.Length Sepal.Width Petal.Length
# 1      7.1      3.0      5.9
# 2      6.5      3.2      5.1
# 3      5.8      2.7      5.1
# ... Species: virginica
```

Step by step

- Specify the class column in `bal_subsampling("Species")`
- The method reduces **all** classes to the size of the minority (11 instances)
- Sampling is performed **without replacement**
- Result: a smaller dataset, but perfectly balanced

📌 **Key concept:** Undersampling discards data from majority classes. It is fast, but may lose relevant information — use it when the dataset is large enough to absorb the loss. With this, we close the cycle on balancing: from the theoretical concept to practical examples, covering both complementary strategies in `daltoolbox` — the topic is now fully addressed.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

